

Interfacing of 8085 with 8255 Programmable Peripheral Interface

Basic: The 8255 is a programmable peripheral interface. It is an IC used to simplify the interfacing of microprocessors and microcontrollers with I/O devices and increase the number of I/O devices that can be interfaced. The best part is that it is programmable, which means that we don't have to change circuit connections and wiring in order to switch between different ports and modes of operation.

Objective:

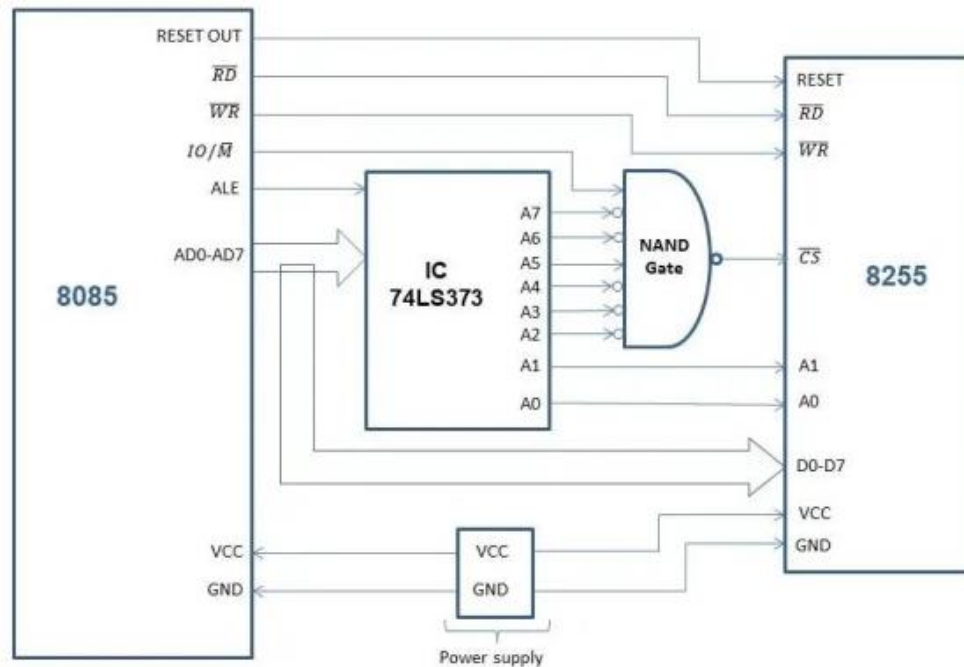
1. We will be covering the interfacing of 8255 with the 8085 microprocessor.
2. We will take up a simple example of taking two numbers as inputs, adding them, and then sending the result to the output port.

Interfacing of 8255 with the 8085 microprocessor

Components required

- 8085 microprocessor
- IC 74LS373 (For address demultiplexing)
- NAND Gate

Circuit diagram to connect the 8085 with 8255



We can understand the connections in the following steps.

- First of all, we need supply power to both 8085 and 8255 by connecting VCC (Voltage common collector) and GND pins to the appropriate sources.
- RESET OUT pin of 8085 is connected to the RESET pin of 8255. When the microprocessor is reset, it also resets the 8255 via this connection.
- $\overline{RD}$ and $\overline{WR}$ pins of 8085 are connected to the $\overline{RD}$ and $\overline{WR}$ pins of 8255, respectively. 8085 conveys the type of operation (read or write) to 8255 via these connections.
- As we have come to know in this 8085 course, pins AD0-AD7 have dual functionality. They act as both; address bus and data bus. This is achieved by multiplexing. To extract address from it, we need to **demultiplex** it, which is achieved using the **IC 74LS373**.
- After demultiplexing, pins A0 and A1 (from the output of IC74LS373) are connected to the pins A0 and A1 of 8255, respectively. These two pins tell 8255 about which port the microprocessor is talking about.
- The last part of making the connections is the chip select logic.

Chip select logic

For the microprocessor to be able to communicate with 8255, the $\overline{CS}$ (active low chip select signal) should be low. So, we design a logic circuit that takes address lines A2-A7 as inputs and $\overline{CS}$ signal as output. The design should be such that if the address allotted to any of the ports A, B, or C appears on A0-A1, output $\overline{CS}$ should go low.

Let us allot the address to the ports A, B, and C of 8255.

Table 11-3: 8255 Port Selection

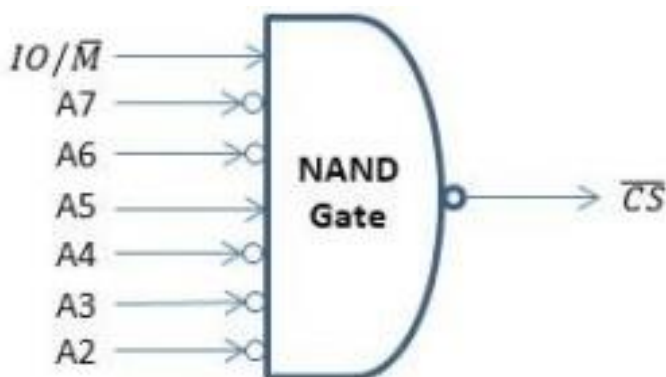
CS	A1	A0	Selects
0	0	0	Port A
0	0	1	Port B
0	1	0	Port C
0	1	1	Control Register
1	x	x	8255 is not selected

- Port A: 0010 0000 (20H)
- Port B: 0010 0001 (21H)
- Port C: 0010 0010 (22H)
- Control port: 0010 0011 (23H)

The last two bits are A0 and A1. The rest of the 6 bits are used to generate chip select signals.

From the above address allocations, we can see that $\overline{CS}$ should be low only when **A7 = A6 = A4 = A3 = A2 = 0** and **A5 = 1**.

Since we are using the NAND gate to implement chip select logic so, the final circuit of the chip select logic will be 0.

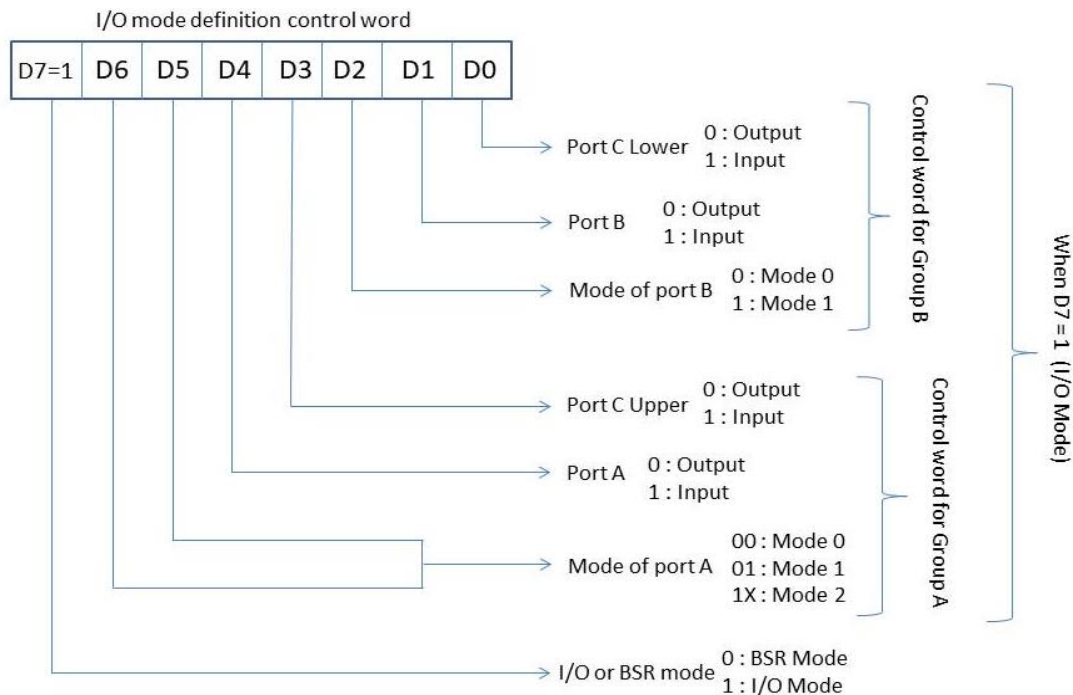


The **NAND** gate outputs **1** unless all its inputs are **1**. So, to get $\overline{CS} = 0$, all the inputs to the **NAND** gate must be **1**.

❖ Consider this example where we are required to take input data from port A and port B and output it at port C.

Step 1

Specify the control word and send it to the control port. The control word of 8255 in I/O mode is shown in the illustration below.



- Ports will be working in I/O mode and not in BSR mode. So, **D7=1**.
- This is a simple I/O operation. So, we need all three ports A, B, and C to work in mode 0. So, **D6 = D5 = D2 = 0**.
- Port A and B are in input mode and port C is in output mode. Therefore, **D4 = D1 = 1** and **D3 = D0 = 0**.

The control word would be 1001 0010 in binary and 92H in hexadecimal. Now, we write this data to the control port to set the port into desired modes. This completes step 1.

Step 2

Read input from port A. Input will be stored in the accumulator register (ax). But we need to read another input from port B, and hence, we move this data to the another register.

Step 3

Read input from port B.

Step 4

Now, we have the two numbers to be added in Accumulator and register. We add both of them and output the value to port C.